Image archives

The Python Imaging Library is ideal for image archival and batch processing applications. You can use the library to create thumbnails, convert between file formats, print images, etc.

The current version identifies and reads a large number of formats. Write support is intentionally restricted to the most commonly used interchange and presentation formats.

Image display

The current release includes Tk *PhotoImage* and *BitmapImage* interfaces, as well as a *Windows DIB interface* that can be used with PythonWin and other Windows-based toolkits. Many other GUI toolkits come with some kind of PIL support.

For debugging, there's also a *show()* method which saves an image to disk, and calls an external display utility.

Image processing

The library contains basic image processing functionality, including point operations, filtering with a set of built-in convolution kernels, and colour space conversions.

The library also supports image resizing, rotation and arbitrary affine transforms.

There's a histogram method allowing you to pull some statistics out of an image. This can be used for automatic contrast enhancement, and for global statistical analysis.

1.2.2 Tutorial

Using the Image class

The most important class in the Python Imaging Library is the *Image* class, defined in the module with the same name. You can create instances of this class in several ways; either by loading images from files, processing other images, or creating images from scratch.

To load an image from a file, use the *open()* function in the *Image* module:

```
>>> from PIL import Image
>>> im = Image.open("hopper.ppm")
```

If successful, this function returns an *Image* object. You can now use instance attributes to examine the file contents:

```
>>> print(im.format, im.size, im.mode)
PPM (512, 512) RGB
```

The *format* attribute identifies the source of an image. If the image was not read from a file, it is set to None. The size attribute is a 2-tuple containing width and height (in pixels). The *mode* attribute defines the number and names of the bands in the image, and also the pixel type and depth. Common modes are "L" (luminance) for grayscale images, "RGB" for true color images, and "CMYK" for pre-press images.

If the file cannot be opened, an `OSError` exception is raised.

Once you have an instance of the *Image* class, you can use the methods defined by this class to process and manipulate the image. For example, let's display the image we just loaded:

```
>>> im.show()
```

(continued from previous page)

```
    if infile != outfile:
        try:
            with Image.open(infile) as im:
                im.thumbnail(size)
                im.save(outfile, "JPEG")
        except OSError:
            print("cannot create thumbnail for", infile)
```



It is important to note that the library doesn't decode or load the raster data unless it really has to. When you open a file, the file header is read to determine the file format and extract things like mode, size, and other properties required to decode the file, but the rest of the file is not processed until later.

This means that opening an image file is a fast operation, which is independent of the file size and compression type. Here's a simple script to quickly identify a set of image files:

Identify image files

```
import sys
from PIL import Image

for infile in sys.argv[1:]:
    try:
        with Image.open(infile) as im:
            print(infile, im.format, f"{im.size}x{im.mode}")
    except OSError:
        pass
```

Cutting, pasting, and merging images

The *Image* class contains methods allowing you to manipulate regions within an image. To extract a sub-rectangle from an image, use the *crop()* method.

Copying a subrectangle from an image

```
box = (0, 0, 64, 64)
region = im.crop(box)
```

The region is defined by a 4-tuple, where coordinates are (left, upper, right, lower). The Python Imaging Library uses a coordinate system with (0, 0) in the upper left corner. Also note that coordinates refer to positions between the pixels, so the region in the above example is exactly 64x64 pixels.

The region could now be processed in a certain manner and pasted back.

`Resampling.`**`BILINEAR`**

For resize, calculate the output pixel value using linear interpolation on all pixels that may contribute to the output value. For other transformations linear interpolation over a 2x2 environment in the input image is used.

`Resampling.`**`HAMMING`**

Produces a sharper image than *Resampling.BILINEAR*, doesn't have dislocations on local level like with *Resampling.BOX*. This filter can only be used with the *resize()* and *thumbnail()* methods.

Added in version 3.4.0.

`Resampling.`**`BICUBIC`**

For resize, calculate the output pixel value using cubic interpolation on all pixels that may contribute to the output value. For other transformations cubic interpolation over a 4x4 environment in the input image is used.

`Resampling.`**`LANCZOS`**

Calculate the output pixel value using a high-quality Lanczos filter (a truncated sinc) on all pixels that may contribute to the output value. This filter can only be used with the *resize()* and *thumbnail()* methods.

Added in version 1.1.3.

`Resampling.`**`MKS2013`**

Calculate the output pixel value using the Magic Kernel Sharp 2013 filter (a quadratic B-spline composed with a sharpening kernel) on all pixels that may contribute to the output value. This filter can only be used with the *resize()* and *thumbnail()* methods.

Added in version 13.0.0.

`Resampling.`**`MKS2021`**

Calculate the output pixel value using the Magic Kernel Sharp 2021 filter (a quadratic B-spline composed with a sharpening kernel) on all pixels that may contribute to the output value. This filter can only be used with the *resize()* and *thumbnail()* methods.

Added in version 13.0.0.

Filters comparison table

Filter	Downscaling quality	Upscaling quality	Performance
Resampling.NEAREST			
Resampling.BOX			
Resampling.BILINEAR			
Resampling.HAMMING			
Resampling.BICUBIC			
Resampling.LANCZOS			
Resampling.MKS2013			
Resampling.MKS2021			

1.2.4 Appendices

Returns
An *Image* object.

Raises

- **FileNotFoundError** – If the file cannot be found.
- ***PIL.UnidentifiedImageError*** – If the image cannot be opened and identified.
- **ValueError** – If the `mode` is not "r", or if a `StringIO` instance is used for `fp`.
- **TypeError** – If `formats` is not `None`, a list or a tuple.

> **Warning**
>
> To protect against potential DOS attacks caused by "decompression bombs" (i.e. malicious files which decompress into a huge amount of data and are designed to crash or cause disruption by using up a lot of memory), Pillow will issue a `DecompressionBombWarning` if the number of pixels in an image is over a certain limit, *MAX_IMAGE_PIXELS*.
>
> This threshold can be changed by setting *MAX_IMAGE_PIXELS*. It can be disabled by setting `Image.MAX_IMAGE_PIXELS = None`.
>
> If desired, the warning can be turned into an error with `warnings.simplefilter('error', Image.DecompressionBombWarning)` or suppressed entirely with `warnings.simplefilter('ignore', Image.DecompressionBombWarning)`. See also the logging documentation to have warnings output to the logging facility instead of stderr.
>
> If the number of pixels is greater than twice *MAX_IMAGE_PIXELS*, then a `DecompressionBombError` will be raised instead.

Image processing

PIL.Image.**alpha_composite**(*im1:* Image, *im2:* Image) → *Image*

Alpha composite im2 over im1.

Parameters

- **im1** – The first image. Must have mode RGBA or LA.
- **im2** – The second image. Must have the same mode and size as the first image.

Returns
An *Image* object.

PIL.Image.**blend**(*im1:* Image, *im2:* Image, *alpha:* *float*) → *Image*

Creates a new image by interpolating between two input images, using a constant alpha:

```
out = image1 * (1.0 - alpha) + image2 * alpha
```

Parameters

- **im1** – The first image.
- **im2** – The second image. Must have the same mode and size as the first image.
- **alpha** – The interpolation alpha factor. If alpha is 0.0, a copy of the first image is returned. If alpha is 1.0, a copy of the second image is returned. There are no restrictions on the alpha value. If necessary, the result is clipped to fit into the allowed output range.

PIL.ImageOps.**posterize**(*image:* Image, *bits:* *int*) → *Image*

Reduce the number of bits for each color channel.

Parameters

- **image** – The image to posterize.
- **bits** – The number of bits to keep for each channel (1-8).

Returns
An image.

PIL.ImageOps.**solarize**(*image:* Image, *threshold:* *int* = *128*) → *Image*

Invert all pixel values above a threshold.

Parameters

- **image** – The image to solarize.
- **threshold** – All pixels above this grayscale level are inverted.

Returns
An image.

PIL.ImageOps.**exif_transpose**(*image:* Image, *, *in_place: Literal[True]*) → None

PIL.ImageOps.**exif_transpose**(*image:* Image, *, *in_place: Literal[False] = False*) → *Image*

If an image has an EXIF Orientation tag, other than 1, transpose the image accordingly, and remove the orientation data.

Parameters

- **image** – The image to transpose.
- **in_place** – Boolean. Keyword-only argument. If `True`, the original image is modified in-place, and `None` is returned. If `False` (default), a new *Image* object is returned with the transposition applied. If there is no transposition, a copy of the image will be returned.

Resize relative to a given size

```
from PIL import Image, ImageOps
size = (100, 150)
with Image.open("Tests/images/hopper.webp") as im:
    ImageOps.contain(im, size).save("imageops_contain.webp")
    ImageOps.cover(im, size).save("imageops_cover.webp")
    ImageOps.fit(im, size).save("imageops_fit.webp")
    ImageOps.pad(im, size, color="#f00").save("imageops_pad.webp")

    # thumbnail() can also be used,
    # but will modify the image object in place
    im.thumbnail(size)
    im.save("image_thumbnail.webp")
```

	thumbnail()	*contain()*	*cover()*	*fit()*	*pad()*
Given size	(100, 150)	(100, 150)	(100, 150)	(100, 150)	(100, 150)
Resulting image					
Resulting size	100×100	100×100	150×150	100×150	100×150